

COURSE HANDOUT [PBL REBUILD — CORRECT VERSION]

Mathematics for Communication Systems (MFCS)

B.Tech ECE | Trimester 02 | Credits: 4 | Contact Hours: 6 | Pre-Requisite: DMS | Program: ECE only

COURSE VISION — PBL-CORRECT BOTTOM-UP PHILOSOPHY

The PBL Inversion Corrected: The original MFCS course opened with signal representation theory before the BPSK system existed as context. This rebuild opens with the complete CommLink-BPSK system running on Day 1. Students watch it transmit 10,000 bits and produce a BER curve. Then the faculty removes layer 1 and asks: 'How do you represent the transmitted signal mathematically?' Each module is entered only when rebuilding the current layer of CommLink requires a mathematical concept not yet in the student's toolkit.

CO Sequence = CommLink Build Sequence: CO1 = Layer 1: signal representation (how to write the BPSK waveform). CO2 = Layer 2: Fourier Transform (what frequencies does it occupy). CO3 = Layer 3: LTI + matched filter (how the channel distorts it, how the receiver recovers it). CO4 = Layer 4: Z-transform + FIR (how to implement the matched filter digitally). CO5 = Layer 5: probability + BER (how many errors to expect). CO6 = Layer 6: Shannon capacity (how close is BPSK to the theoretical limit). Each layer is introduced only when the prior layer cannot be completed without it.

Main Project Outcome: CommLink-BPSK — a complete Jupyter Notebook implementing a BPSK digital communication system through 6 mathematical layers, achieving $BER = Q(\sqrt{2E_b/N_0})$ verified by Monte Carlo simulation agreeing with the theoretical formula to within 20% at all SNR values, and computing the Shannon capacity gap (11.2 dB at $BER=10^{-5}$) between BPSK and the theoretical limit.

TABLE 1 — COURSE OUTCOMES — EACH CO = ONE COMMLINK BUILD LAYER

The 'CommLink Barrier' column specifies the exact question a student cannot answer while building the current CommLink layer without the new mathematical concept. CO6 (Shannon information theory) is last — it is the deepest foundation, revealed only after the complete system is built.

CO #	BTL	Course Outcome Statement	POs	CommLink Barrier That Introduces This CO	Assessment
CO1	BTL 3 (Apply)	Apply signal representation — sinusoids, amplitude, frequency, phase, superposition, energy and power of periodic signals, and phasor notation — to describe the physical waveforms that CommLink-BPSK uses to transmit binary data, introduced when students attempt to represent a '1' bit as a voltage waveform and discover they need a precise mathematical language for time-domain signals.	PO1,PO2,PO3	CommLink barrier 1: students try to describe the BPSK '1' signal as 'a positive wave' and the '0' as 'a negative wave' — but cannot write either as a function of time without sinusoidal signal representation. Cannot compute signal energy without integration. Cannot specify the carrier frequency without the mathematical definition of frequency.	Signal representation derivation (hand) + BPSK waveform specification + energy/power computation
CO2	BTL 4 (Analyse)	Analyse the Fourier Series and Fourier Transform — deriving Fourier coefficients, the Fourier integral, the spectrum of a BPSK signal, Parseval's theorem, and convolution in the frequency domain — introduced when students have a time-domain BPSK signal but the receiver cannot separate it from noise without knowing what frequencies the signal occupies and whether any overlap with the noise spectrum.	PO1,PO2,PO3	CommLink barrier 2: the BPSK signal is transmitted. At the receiver, noise is added. Students ask: 'can we filter out the noise without distorting the signal?' To answer this, they need to know what frequencies the BPSK signal occupies — the spectrum. The Fourier	Fourier Transform derivation (hand) + BPSK spectrum analysis + Parseval's theorem verification

CO #	BTL	Course Outcome Statement	POs	CommLink Barrier That Introduces This CO	Assessment
				Transform is the tool that reveals the spectrum.	
CO3	BTL 4 (Analyse)	Analyse Linear Time-Invariant (LTI) systems — impulse response, convolution, the system's frequency response $H(f)=Y(f)/X(f)$, and the matched filter derivation — introduced when students model the physical channel as a system that modifies the transmitted BPSK signal, and discover they need a mathematical framework to describe what ANY linear system does to ANY input signal.	PO1,PO2,PO3	CommLink barrier 3: the physical channel is not a perfect wire — it attenuates some frequencies more than others (a low-pass channel). The received signal is distorted. Students ask: 'how do I model what the channel does?' LTI convolution is the answer. The matched filter — the optimal receiver filter for BPSK — requires convolving with the time-reversed signal.	LTI convolution derivation + matched filter design + channel distortion analysis using Python/SciPy
CO4	BTL 4 (Analyse)	Analyse the Z-transform and digital filter design — the Z-transform pair, region of convergence, transfer function $H(z)$, poles and zeros, FIR and IIR filter design — introduced when students implement the matched filter in software and discover that continuous-time convolution must be discretised for digital implementation, requiring a transform that handles sampled signals.	PO1,PO2,PO3	CommLink barrier 4: the matched filter is designed in continuous time, but CommLink-BPSK is implemented in Python — it operates on discrete samples. The Z-transform is the discrete-time equivalent of the Laplace/Fourier transform. Students need it to convert the matched filter design to a discrete-time FIR filter they can implement.	Z-transform derivation + FIR matched filter implementation + Python/SciPy digital filter verification
CO5	BTL 5 (Evaluate)	Evaluate probability and the Bit Error Rate derivation — Gaussian noise model, conditional probability, the Q-function as the tail probability of the Gaussian, and the BPSK BER formula $P_e = Q(\sqrt{2E_b/N_0})$ — introduced when students run CommLink-BPSK and observe bit errors, and need to predict how many errors to expect at a given signal-to-noise ratio and verify their implementation is correct.	PO1,PO2,PO3	CommLink barrier 5: CommLink-BPSK transmits 10,000 bits. Some are received incorrectly. Students ask: 'how many errors should we expect?' and 'how much does increasing transmit power reduce errors?' These questions require: the Gaussian noise model, conditional probability of a decision error, and the Q-function BER formula.	AWGN model + Q-function derivation + BER formula $P_e=Q(\sqrt{2E_b/N_0})$ + Monte Carlo BER vs. theoretical BER
CO6	BTL 5 (Evaluate)	Evaluate Shannon's channel capacity theorem and information theory — entropy, mutual information, channel capacity $C=B \log_2(1+SNR)$, and the Shannon limit — as the deepest mathematical foundation of the entire CommLink system, introduced last when students ask the most fundamental question: 'is BPSK a good modulation scheme, or are there fundamentally better ones?', requiring the theory that establishes the theoretical maximum rate of error-free communication.	PO1,PO2,PO3	CommLink barrier 6 (the revelation): students have built a working system achieving $P_e = Q(\sqrt{2E_b/N_0})$. They ask: 'can any modulation scheme do better than this?' Shannon's theorem answers: yes, if you use a more complex code, but there is a hard limit $C = B \log_2(1+SNR)$ beyond which error-free communication is	Shannon capacity derivation + entropy + mutual information + CommLink spectral efficiency vs. Shannon limit

CO #	BTL	Course Outcome Statement	POs	CommLink Barrier That Introduces This CO	Assessment
				impossible. BPSK's efficiency = its spectral efficiency vs. the Shannon limit.	

TABLE 2 — SYLLABUS — EACH MODULE OPENS WITH THE COMMLINK BARRIER THAT DEMANDS IT

Every module opens with 'CommLink barrier' — the specific engineering question that cannot be answered without the new mathematics. Signal processing mathematics is introduced as the tool that makes the next CommLink layer buildable.

Module	CO	Module Title	Topics — each introduced only after CommLink hits the barrier that demands them
Module 1	CO1	Signal Representation — Writing the BPSK Waveform as a Mathematical Function	CommLink barrier: 'I want to represent a binary '1' as a physical signal. My first attempt: say it's a 'positive voltage'. But a receiver must sample this signal at precise time instants. A constant DC voltage carries no information about timing. A '1' must be distinguishable from a '0' in a precise mathematical way. I need to describe the signal as a function of time — and that means sinusoids.' 1.1 The problem with 'positive voltage': demonstrating that a constant DC signal cannot be recovered from a noisy channel — the sinusoidal carrier enables coherent detection at the receiver 1.2 Sinusoids: $x(t) = A \cos(2\pi f t + \phi)$ — amplitude A, frequency f (Hz), angular frequency $\omega = 2\pi f$ (rad/s), phase ϕ — each parameter identified in the BPSK context: A=signal amplitude, f=carrier frequency (e.g. 1 MHz), $\phi=0$ for bit '1', $\phi=\pi$ for bit '0' 1.3 BPSK signal definition: $s_1(t) = A \cos(2\pi f_c t)$ for bit '1', $s_0(t) = A \cos(2\pi f_c t + \pi) = -A \cos(2\pi f_c t)$ for bit '0' — the two signals differ only in phase — antipodal signalling 1.4 Superposition principle: $x(t) = A \cos(2\pi f_1 t) + B \cos(2\pi f_2 t)$ — demonstrating that signals add linearly — this will be the mathematical basis of the Fourier series (sum of sinusoids) 1.5 Energy of a signal: $E = \int_0^T x^2(t) dt$ — computing the energy of the BPSK symbol $s_1(t)$ over one bit period T_b — result: $E_b = A^2 T_b / 2$ — the energy per bit is determined by amplitude and bit duration 1.6 Power of a signal: $P = (1/T) \int_0^T x^2(t) dt$ — average power of the BPSK signal — $P = A^2 / 2$ — connecting to the transmit power specification in CommLink 1.7 Phasor representation: $x(t) = \text{Re}\{A e^{j(2\pi f t + \phi)}\}$ — Euler's formula $e^{j\theta} = \cos(\theta) + j \sin(\theta)$ — the phasor $A e^{j\phi}$ compactly encodes amplitude and phase — the BPSK constellation: two phasors at angles 0 and π on the real axis 1.8 Orthogonality: two sinusoids $x_1(t) = \cos(2\pi f_1 t)$ and $x_2(t) = \cos(2\pi f_2 t)$ are orthogonal over $[0, T]$ if $f_1 \neq f_2$ and both are integer multiples of $1/T$ — $\int_0^T x_1(t)x_2(t) dt = 0$ — this property enables separating signals at the receiver 1.9 Sampling theorem preview: to recover a signal of bandwidth B Hz, the receiver must sample at least $2B$ samples per second — motivating Module 4's discrete-time treatment 1.10 CommLink Layer 1 complete: students write the complete mathematical specification of both BPSK symbols — amplitude, frequency, phase, energy per bit E_b — these four quantities will appear in every subsequent module
Module 2	CO2	Fourier Transform — Revealing What Frequencies the BPSK Signal Occupies	CommLink barrier: 'My BPSK signal is transmitted over a channel that has limited bandwidth — it attenuates signals above 10 kHz. My carrier is at 1 MHz. Will the channel distort my signal? To answer this, I need to know what frequencies my BPSK signal actually contains. The Fourier Transform tells me the spectrum of any signal.' 2.1 From sinusoid to spectrum: a single sinusoid $A \cos(2\pi f_0 t)$ contains exactly one frequency — its spectrum is a single impulse at $f=f_0$ — but a BPSK signal switches between phases, creating transitions that generate sidebands 2.2 The Fourier Series: for a periodic signal $x(t)$ with period $T_0 = 1/f_0$: $x(t) = \sum_n c_n e^{j2\pi n f_0 t}$ — the Fourier coefficient $c_n = (1/T_0) \int_0^{T_0} x(t) e^{-j2\pi n f_0 t} dt$ — derivation using orthogonality of complex exponentials 2.3 Computing the Fourier Series of a square wave: the BPSK bit stream is (locally) a square wave — $c_n = A \sin(n\pi/2)/(n\pi/2)$ (sinc form) — infinite bandwidth: square wave transitions require infinitely many harmonics 2.4 The Fourier Transform: for aperiodic signals — $X(f) = \int_{-\infty}^{\infty} x(t) e^{-j2\pi f t} dt$ — inverse: $x(t) = \int_{-\infty}^{\infty} X(f) e^{j2\pi f t} df$ — the spectrum $X(f)$ as a function of continuous frequency 2.5 Key Fourier Transform pairs: $\text{rect}(t/T) \leftrightarrow T \text{sinc}(fT)$ — the BPSK rectangular pulse of duration T_b has a sinc-shaped spectrum — bandwidth: first null at $1/T_b$ (null-to-null bandwidth = $2/T_b$) 2.6 Fourier Transform properties: linearity, time shift ($x(t-t_0) \leftrightarrow X(f) e^{-j2\pi f t_0}$), frequency shift ($x(t) e^{j2\pi f_c t} \leftrightarrow X(f-f_c)$) — the bandpass BPSK spectrum is the baseband rectangular pulse spectrum shifted to $\pm f_c$

Module	CO	Module Title	Topics — each introduced only after CommLink hits the barrier that demands them
			2.7 Parseval's theorem: $\int x(t) ^2 dt = \int X(f) ^2 df$ — total energy in time domain equals total energy in frequency domain — used to verify the BPSK energy per bit: $E_b = A^2 T_b / 2$ computed both in time and frequency 2.8 Convolution theorem: $(x*h)(t) = \int x(\tau)h(t-\tau)d\tau$ — Fourier Transform of convolution is multiplication: $F\{x*h\} = X(f) \cdot H(f)$ — the critical insight: filtering in time domain = multiplication in frequency domain 2.9 Power Spectral Density: the PSD of the BPSK signal $S_x(f) = X(f) ^2 / T_b$ (in dB vs. frequency) — plotting the sinc²-shaped main lobe centred at f_c — identifying the 90% bandwidth 2.10 CommLink Layer 2: students compute the PSD of the BPSK signal — determine the null-to-null bandwidth as $2/T_b$ — verify with Python/SciPy FFT that the spectrum matches the derived sinc² shape — answer the channel question: if the channel passes all frequencies up to $f_c + 1/T_b$, the BPSK signal passes with negligible distortion
Module 3	CO3	LTI Systems and Convolution — Modelling the Channel and Designing the Matched Filter	CommLink barrier: 'The physical transmission channel attenuates higher frequencies differently than lower ones. When I receive the BPSK signal, it is not the same shape as what was transmitted — some frequency components are stronger, others weaker. I need a mathematical model that tells me: given a channel with a known frequency response $H(f)$, what is the received signal? And once I know the received signal, how do I design a receiver filter that maximises my ability to detect the bit correctly?' 3.1 The channel model: the AWGN channel: $r(t) = s(t) * h(t) + n(t)$ — the received signal $r(t)$ is the convolution of the transmitted signal with the channel impulse response, plus additive white Gaussian noise $n(t)$ 3.2 LTI system definition: Linear (superposition: if $x_1 \rightarrow y_1$ and $x_2 \rightarrow y_2$, then $ax_1 + bx_2 \rightarrow ay_1 + by_2$) and Time-Invariant (shift: if $x(t) \rightarrow y(t)$, then $x(t-\tau) \rightarrow y(t-\tau)$) — why these two properties make the channel mathematically tractable 3.3 The impulse response $h(t)$: if $\delta(t) \rightarrow h(t)$ (the system's response to a unit impulse) then by linearity and time-invariance: $x(t) \rightarrow (x*h)(t) = \int x(\tau)h(t-\tau)d\tau$ — $h(t)$ completely characterises the LTI system 3.4 Convolution: $(x*h)(t) = \int_{-\infty}^{\infty} x(\tau)h(t-\tau)d\tau$ — geometric interpretation: flipping h, sliding it over x — commutative, associative, distributive — computing the convolution of a rectangular pulse with a Gaussian $h(t)$ by hand 3.5 Frequency domain analysis: the received spectrum is $R(f) = S(f) \cdot H(f) + N(f)$ — the channel in the frequency domain is a multiplication by $H(f)$ (the channel transfer function) — a low-pass channel with $H(f) = \text{rect}(f/B)$ passes only signals within bandwidth B 3.6 The BPSK channel model: assuming a flat (constant $H(f)$) AWGN channel for CommLink simplicity — the received signal is $r(t) = \pm A \cos(2\pi f_c t) + n(t)$ — the noise $n(t)$ has PSD $N_0/2$ W/Hz 3.7 The matched filter: the receiver filter that maximises the SNR at the sampling instant — the matched filter to signal $s(t)$ has impulse response $h_{mf}(t) = s(T_b - t)$ (time-reversed, delayed version of the transmitted signal) 3.8 Derivation of the matched filter: the SNR at time $t=T_b$ after the matched filter is $\text{SNR} = 2E_b/N_0$ — this is the maximum achievable SNR for any linear filter — the Cauchy-Schwarz inequality is the tool used in the derivation 3.9 The correlator receiver: computing the matched filter output as a cross-correlation — $\int_0^{T_b} r(t) \cdot s(t) dt$ — for BPSK: this simplifies to measuring the sign of $\int_0^{T_b} r(t) \cdot \cos(2\pi f_c t) dt$ 3.10 Python/SciPy implementation: simulate the BPSK channel — generate 1000 BPSK symbols, add AWGN noise, apply the correlator receiver using <code>numpy.correlate</code> — verify that the decision threshold at 0 is optimal 3.11 CommLink Layer 3: students implement the channel model and matched filter correlator in Python — test at $E_b/N_0 = 5$ dB — count bit errors — the error rate will be computed theoretically in Module 5
Module 4	CO4	Z-Transform and Digital Filter Design — Implementing the Matched Filter in Discrete Time	CommLink barrier: 'My matched filter is designed as a continuous-time integral. But CommLink-BPSK is a Python simulation — it operates on discrete samples at rate f_s samples per second. I need to implement the matched filter as a discrete-time filter. The Z-transform is the discrete-time equivalent of the Laplace/Fourier Transform, and it tells me how to design a digital filter from a system specification.'

Module	CO	Module Title	Topics — each introduced only after CommLink hits the barrier that demands them
			4.1 Discrete-time signals: sampling $x(t)$ at rate f_s produces $x[n] = x(nT_s)$ where $T_s = 1/f_s$ — the Nyquist-Shannon sampling theorem: must have $f_s \geq 2B$ to avoid aliasing — aliasing demonstration for BPSK sampled at sub-Nyquist rate 4.2 The Z-transform: $X(z) = \sum_n x[n] z^{-n}$ — the bilateral Z-transform — the unit circle $z = e^{j\omega}$ connects Z-transform to DTFT: $X(e^{j\omega})$ is the discrete-time Fourier Transform (DTFT) 4.3 Key Z-transform pairs: $x[n] = \delta[n] \leftrightarrow X(z) = 1$, $x[n] = u[n] \leftrightarrow 1/(1-z^{-1})$, $x[n] = a^n u[n] \leftrightarrow 1/(1-az^{-1})$ — deriving each from the definition 4.4 Z-transform properties: linearity, time delay ($x[n-k] \leftrightarrow z^{-k}X(z)$), convolution ($x*h \leftrightarrow X(z) \cdot H(z)$) — the discrete-time convolution theorem mirrors the continuous-time version 4.5 Region of Convergence (ROC): the ROC of $X(z) = \sum x[n]z^{-n}$ is the set of z for which the sum converges — causal sequences (ROC is outside a circle), anticausal sequences (inside a circle), two-sided sequences (annular ROC) 4.6 Poles and zeros: $H(z) = B(z)/A(z)$ — zeros: roots of $B(z)$ — poles: roots of $A(z)$ — the frequency response on the unit circle $H(e^{j\omega})$ is shaped by the proximity of poles (amplify) and zeros (attenuate) to the unit circle 4.7 FIR filter design: $h[n] = \{h_0, h_1, \dots, h_{N-1}\}$ — all poles at origin — $H(z) = \sum_n h_n z^{-n}$ — always stable — linear phase if $h[n] = h[N-1-n]$ (symmetric coefficients) — the discrete matched filter for BPSK: $h[n] = s[N-1-n]$ (time-reversed transmitted pulse samples) 4.8 IIR filter design: $H(z) = B(z)/A(z)$ with non-trivial denominator — poles can be anywhere inside the unit circle — more efficient than FIR for the same frequency response — but can be unstable if poles are outside the unit circle 4.9 Digital filter implementation in Python: <code>scipy.signal.lfilter(b, a, x)</code> — b = FIR numerator coefficients, a = denominator — implementing the BPSK matched filter as an FIR convolution: <code>numpy.convolve(received_signal, time_reversed_pulse)</code> 4.10 CommLink Layer 4: students replace the continuous-time correlator with a Python FIR matched filter using <code>numpy.convolve</code> — verify the output at the optimal sampling instant is equivalent to the correlator — test at multiple SNR values: BER should be the same
Module 5	CO5	Probability and BER — Deriving How Many Bit Errors BPSK Should Make	CommLink barrier: 'After running CommLink-BPSK for 10,000 bits at $E_b/N_0 = 6$ dB, I observe 79 bit errors — a BER of 0.79%. Is this correct? Too many? Too few? To know, I need to derive the theoretical BER for BPSK analytically. This requires: modelling the noise as a Gaussian random variable, computing the conditional probability of a decision error, and evaluating the Gaussian tail probability.' 5.1 The AWGN noise model: $n(t)$ is white Gaussian noise with two-sided PSD $N_0/2$ W/Hz — after the matched filter and sampler, the noise sample n is a Gaussian random variable $N(0, N_0/2)$ — why Gaussian: by CLT, the sum of many independent noise contributions is Gaussian 5.2 The received sample model: for transmitted bit '1', the matched filter output at $t=T_b$ is $y = +\sqrt{E_b} + n$ where $n \sim N(0, N_0/2)$ — for bit '0': $y = -\sqrt{E_b} + n$ — two Gaussian distributions shifted by $\pm\sqrt{E_b}$ 5.3 The decision rule: decide '1' if $y > 0$, decide '0' if $y < 0$ — the optimal decision threshold is 0 (midpoint between the two signal points) — proof: this minimises P_e when bits are equally likely 5.4 Conditional probability of error for bit '1': $P(\text{error} '1') = P(y < 0 '1') = P(\sqrt{E_b} + n < 0) = P(n < -\sqrt{E_b})$ — for $n \sim N(0, N_0/2)$ 5.5 The Q-function: $Q(x) = P(Z > x)$ for $Z \sim N(0,1) = \int_x^\infty \frac{1}{\sqrt{2\pi}} e^{-t^2/2} dt$ — the Gaussian tail probability — cannot be computed in closed form — computed numerically 5.6 BPSK BER derivation: $P(\text{error} '1') = P(n < -\sqrt{E_b}) = P(n/\sqrt{N_0/2} < -\sqrt{(2E_b/N_0)}) = Q(\sqrt{(2E_b/N_0)})$ — by symmetry: $P(\text{error} '0') = Q(\sqrt{(2E_b/N_0)})$ — total BER: $P_e = Q(\sqrt{(2E_b/N_0)})$ 5.7 The BER curve: plotting P_e vs. E_b/N_0 in dB — at $E_b/N_0 = 6$ dB: $P_e \approx 10^{-2}$ — at 10 dB: $P_e \approx 10^{-6}$ — every 3 dB increase in E_b/N_0 approximately halves the number of errors — the waterfall region

Module	CO	Module Title	Topics — each introduced only after CommLink hits the barrier that demands them
			5.8 Monte Carlo BER simulation: Python — generate $N=100,000$ BPSK bits — transmit through AWGN channel at $E_b/N_0 = 0, 2, 4, 6, 8, 10$ dB — apply matched filter — count errors — plot simulated BER vs. theoretical Q-function — they must agree within statistical noise 5.9 The theoretical vs. simulated BER comparison: the match verifies that the CommLink implementation is mathematically correct — any deviation indicates a bug in the noise generation, filter, or sampling 5.10 CommLink Layer 5 complete: students produce the BER curve plot with both theoretical $P_e = Q(\sqrt{2E_b/N_0})$ and simulated BER on the same axes — for $E_b/N_0 = 6$ dB: theoretical $P_e \approx 0.0079$ — verifying the 79 errors in 10,000 bits are statistically consistent 5.11 Module barrier forward: BPSK achieves $P_e = Q(\sqrt{2E_b/N_0})$. Students ask: 'Is this good? Is it the best possible? Could a different modulation scheme do better?' This question cannot be answered within the signal processing framework — it requires a completely different mathematical theory. Module 6 arrives: Shannon's theorem.
Module 6	CO6	Shannon Capacity and Information Theory — The Fundamental Limit BPSK Approaches	Revelation barrier: 'CommLink-BPSK works. It achieves $BER = Q(\sqrt{2E_b/N_0})$. But how good is this really? Could a completely different modulation scheme achieve $BER = 10^{-6}$ at lower power? And is there any limit to how reliably we can communicate? Shannon's theorem answers all of these questions — and it was derived purely from probability theory, without any reference to specific modulation schemes. It is the mathematical foundation that explains why communication systems look the way they do.' 6.1 The information problem: Shannon's question (1948) — given a noisy channel, what is the maximum rate at which information can be transmitted with arbitrarily small probability of error? 6.2 Information content: the information in an event with probability p is $h(p) = -\log_2(p)$ bits — a certain event ($p=1$) carries 0 bits of information — a rare event (p very small) carries many bits 6.3 Entropy: $H(X) = -\sum p(x) \log_2 p(x)$ — the average information content of a random variable — derivation of the property that H is maximised when all outcomes are equally likely — $H(X) \leq \log_2 X$ with equality iff X is uniform 6.4 Entropy of the BPSK bit stream: $H(\text{bit}) = -p \cdot \log_2(p) - (1-p) \cdot \log_2(1-p) = 1$ bit (for equally likely bits $p=0.5$) — each bit carries exactly 1 bit of information — this is why BPSK is a 1 bit/symbol modulation scheme 6.5 Mutual information: $I(X;Y) = H(X) - H(X Y)$ — the amount of information Y provides about X — for the BPSK channel: $I(\text{bit}; \text{received sample})$ is the information the noisy received signal provides about the transmitted bit 6.6 Channel capacity: $C = \max_{\{p(x)\}} I(X;Y)$ — the maximum mutual information over all input distributions — Shannon's fundamental result 6.7 Shannon's AWGN capacity theorem: for a channel with bandwidth B Hz and SNR: $C = B \log_2(1 + \text{SNR})$ bits/second — derivation sketch: Gaussian inputs maximise capacity for Gaussian channels (proof by maximum entropy argument) — this is the absolute maximum rate of reliable communication 6.8 The Shannon limit for CommLink: $B=1/T_b$ Hz (BPSK null-to-null bandwidth) — $\text{SNR} = E_b/N_0 \times (R/B)$ where $R=1/T_b$ is the bit rate — for BPSK: spectral efficiency = 1 bit/s/Hz — the Shannon limit for 1 bit/s/Hz requires $E_b/N_0 > -1.6$ dB — BPSK at 10^{-5} BER requires $E_b/N_0 \approx 9.6$ dB — gap to Shannon limit ≈ 11.2 dB 6.9 Modern coding: turbo codes, LDPC codes, polar codes close the gap to the Shannon limit to within 0.1 dB — the practical significance of Shannon's theorem: it tells engineers exactly how much improvement is possible with better coding 6.10 Course synthesis: every module of MFCS contributed one layer to CommLink: CO1 (sinusoidal signal representation) defined what was transmitted, CO2 (Fourier Transform) revealed the spectrum, CO3 (LTI convolution) modelled the channel and designed the matched filter, CO4 (Z-transform) enabled digital implementation, CO5 (probability + Q-function) quantified the error rate, CO6 (Shannon capacity) established the theoretical limit and how close BPSK comes to it — the entire chain from physical signal to information-theoretic limit 6.11 CommLink Layer 6 complete: students compute the Shannon capacity for the CommLink channel at $E_b/N_0=10$ dB — compare BPSK spectral efficiency (1 bit/s/Hz) to the Shannon limit — plot the E_b/N_0 gap — write a 1-page technical

Module	CO	Module Title	Topics — each introduced only after CommLink hits the barrier that demands them
			summary: 'CommLink-BPSK achieves what fraction of the Shannon capacity, and what coding technique would be needed to close the gap?'

TABLE 3 — MAIN PROJECT: COMMLINK-BPSK — 6-LAYER DIGITAL COMMUNICATION SYSTEM

Dimension	Details
Project Title	CommLink-BPSK — A Complete Binary Phase Shift Keying Digital Communication System: Designed Layer by Layer, Each Layer Introduced When the Previous One Cannot Proceed Without New Mathematics
System Specification	Modulation: BPSK (Binary Phase Shift Keying). Carrier: f_c (configurable). Bit rate: R bits/second. Channel: AWGN with noise PSD $N_0/2$. Target: $BER \leq 10^{-5}$ at $E_b/N_0 = 9.6$ dB. Implementation: Python/NumPy/SciPy/Matplotlib. Validation: theoretical $BER P_e = Q(\sqrt{2E_b/N_0})$ vs. Monte Carlo simulation must agree within 10% at all tested SNR values. Shannon capacity gap: compute and report E_b/N_0 gap between BPSK and Shannon limit.
Why CommLink Is Shown Complete on Day 1	Students see the complete working CommLink-BPSK Python simulation on Day 1 — transmitting 10,000 bits, adding noise, recovering bits, producing the BER curve. They observe it working. Then the faculty disassembles it layer by layer, removing each component and asking: 'what mathematics would you need to put this back together?' This is the PBL-correct method: show the system, then ask the student to rebuild it, introducing the mathematics at each layer where the rebuild stalls.
Layer 1 — Signal Representation (CO1 — Module 1)	Define the two BPSK symbols mathematically: $s_1(t) = A \cos(2\pi f_c t)$ for bit '1', $s_0(t) = -A \cos(2\pi f_c t)$ for bit '0'. Specify: carrier frequency $f_c = 10,000$ Hz, bit rate $R = 1,000$ bits/s, symbol duration $T_b = 1/R = 1$ ms, amplitude $A = 1$ V, energy per bit $E_b = A^2 T_b / 2 = 5 \times 10^{-4}$ J. Python: generate $s_1(t)$ and $s_0(t)$ as NumPy arrays. Plot both waveforms. Compute energy numerically and verify against $E_b = A^2 T_b / 2$.
Layer 2 — BPSK Spectrum (CO2 — Module 2)	Compute the PSD of the BPSK signal using the FFT. Python: <code>numpy.fft.fft</code> of a 100-bit BPSK sequence — plot one-sided PSD vs. frequency. Verify: the main lobe is centred at $f_c = 10,000$ Hz, first nulls at $f_c \pm R = 10,000 \pm 1,000$ Hz (null-to-null bandwidth = $2R = 2,000$ Hz). Parseval's theorem verification: compute signal energy in time domain (sum of squared samples $\times T_s$) and in frequency domain (sum of $ X[k] ^2 \times df$) — they must agree within numerical precision.
Layer 3 — Channel and Matched Filter (CO3 — Module 3)	Implement AWGN channel: $r[n] = s[n] + \text{noise}[n]$ where $\text{noise} \sim N(0, \sigma^2)$ and $\sigma^2 = N_0/2$. Implement the correlator receiver: correlate received sequence with the reference $s_1[n] = \cos(2\pi f_c \cdot n/f_s)$ over one bit period. Decision: if correlation output > 0 , decide '1'; else decide '0'. Test at $E_b/N_0 = 6$ dB: generate and detect 1,000 bits — count errors. Python: <code>numpy.correlate</code> for the correlation. Visualise: show received signal $r(t)$ = noisy BPSK waveform, and the correlator output at each bit period.
Layer 4 — Digital Matched Filter (CO4 — Module 4)	Replace the correlator with an FIR matched filter: $h[n] = s_1[N-1-n]$ (time-reversed transmitted pulse). Implement using <code>numpy.convolve(received_signal, matched_filter_coefficients)</code> . Sample the convolution output at $t = N \times T_s$ (end of each bit period). Verify: FIR matched filter and correlator give identical decisions for all 1,000 test bits (they are mathematically equivalent — different implementations of the same operation). Plot: FIR filter frequency response $ H(e^{j\omega}) $ using <code>numpy.fft</code> and verify it is a bandpass filter centred at f_c .
Layer 5 — BER Curve (CO5 — Module 5)	Monte Carlo BER simulation: for $E_b/N_0 = \{0, 2, 4, 6, 8, 10\}$ dB — at each SNR: (1) compute $\sigma^2 = N_0/2 = E_b/(2 \times 10^{\{E_b/N_0\text{ dB}/10\}})$, (2) transmit $N=100,000$ BPSK bits through AWGN channel, (3) apply matched filter, (4) count bit errors, (5) compute $BER = \text{errors}/N$. Theoretical BER: $P_e = \text{scipy.special.erfc}(\text{numpy.sqrt}(10^{\{E_b/N_0\text{ dB}/10\}})) / 2$ (equivalent to $Q(\sqrt{2E_b/N_0})$). Plot: simulated BER (circles) and theoretical BER (solid line) on the same log-scale plot. Target: maximum deviation $< 20\%$ at each SNR point.
Layer 6 — Shannon Capacity Analysis (CO6 — Module 6)	Compute the Shannon capacity of the CommLink channel: $C = B \log_2(1 + \text{SNR})$ where $B = \text{null-to-null BPSK bandwidth} = 2R = 2,000$ Hz and $\text{SNR} = E_b/N_0 \times (R/B) = E_b/N_0/2$. Plot: BPSK spectral efficiency (1 bit/s/Hz horizontal line) vs. the Shannon capacity curve. Find: at what E_b/N_0 does the Shannon capacity curve cross 1 bit/s/Hz? (Answer: -1.59 dB — the theoretical minimum.) Compute the gap: BPSK at $BER=10^{-5}$ requires $E_b/N_0=9.6$ dB — gap to Shannon limit = $9.6 - (-1.59) = 11.19$ dB. Technical summary: what coding gain would be needed to close this gap?
Tools	Python 3.x + NumPy (signal generation, FFT, FIR filtering) + SciPy (<code>scipy.special.erfc</code> for Q-function, <code>scipy.signal</code> for filter analysis) + Matplotlib (BER curve, spectrum plot, signal waveform plot). All work runs in a single Jupyter Notebook with section structure mirroring the 6 CommLink layers.

Dimension	Details
Feasibility	Weeks 1–2: Module 1 + Layer 1 (signal representation + waveform generation). Weeks 3–4: Module 2 + Layer 2 (Fourier Transform + spectrum). Weeks 5–6: Module 3 + Layer 3 (LTI + matched filter correlator). Weeks 7–8: Module 4 + Layer 4 (Z-transform + FIR matched filter). Weeks 9–10: Module 5 + Layer 5 (BER derivation + Monte Carlo). Weeks 11–12: Module 6 + Layer 6 (Shannon capacity + technical summary).
Deliverables	1. CommLink-BPSK Jupyter Notebook: 6 sections (one per layer), each opening with the barrier it solves, implementing the layer in Python, producing the required plot or verification. 2. Hand-written derivation portfolio (photographed): sinusoidal energy integral, Fourier Transform of rectangular pulse, LTI convolution step, matched filter SNR derivation, BPSK BER derivation, Shannon capacity derivation. 3. BER curve plot: simulated vs. theoretical on log-scale, at 6 SNR values. 4. Shannon gap analysis: capacity vs. spectral efficiency plot + computed gap + 1-page summary. 5. Individual viva: faculty selects one CommLink layer — student derives the key mathematical result, explains the barrier that motivated it, demonstrates the Python output.

TABLE 4 — MICRO PROJECTS — DOMAINS SEPARATE FROM COMMLINK

Every micro project uses a domain completely separate from BPSK (audio, square wave, RC circuits, Wi-Fi, QPSK). This ensures the mathematical concepts generalise beyond the CommLink implementation.

Module	CO	Mode	Micro Project	Description
Module 1	CO1	Pen + Python	Audio Signal Representation	Students represent an audio signal (completely different domain from BPSK): $x(t) = 0.8\cos(2\pi \times 440t) + 0.3\cos(2\pi \times 880t)$ (A4 note + second harmonic). Tasks: (1) Identify all parameters (amplitude, frequency, phase) for each component, (2) Compute the total signal energy over one period $T=1/440$ s — show the integral for each term and verify the cross-terms vanish (orthogonality), (3) Verify numerically in Python using <code>numpy.trapz</code> , (4) Plot $x(t)$ for 5 periods in Matplotlib, (5) Phasor representation: draw the phasor diagram for both components at $t=0$, (6) What sampling rate is required to avoid aliasing? (Nyquist: $\geq 2 \times 880 = 1760$ Hz). Domain: audio engineering, completely separate from CommLink.
Module 2	CO2	Pen + Python	Square Wave Spectrum Analysis	Students analyse a perfect square wave $x(t) = \text{sign}(\sin(2\pi f_0 t))$ with $f_0=100$ Hz (completely different from CommLink rectangular pulse): (1) Derive the first 5 non-zero Fourier Series coefficients analytically — show the integral $\int_0^T x(t)e^{-j2\pi n f_0 t} dt$ for $n=1,3,5,7,9$, (2) Verify by reconstructing $x(t)$ from the first 5 harmonics: $x_5(t) = \sum c_n e^{j2\pi n f_0 t}$ — plot $x_5(t)$ vs. true $x(t)$, observe Gibbs phenomenon, (3) Python: <code>numpy.fft.fft</code> on 1024-sample square wave — plot the spectrum — verify the theoretical harmonic amplitudes match, (4) Parseval verification: sum of $ c_n ^2$ from the formula must equal the average power of the square wave ($P=1$ for a unit square wave). Domain: signal analysis, separate from CommLink.
Module 3	CO3	Pen + Python	RC Low-Pass Filter Analysis	Students analyse a first-order RC low-pass filter (completely different from the matched filter): (1) Derive the impulse response $h(t) = (1/RC)e^{-t/RC}u(t)$ from the circuit equation $dV_{\text{out}}/dt + V_{\text{out}}/RC = V_{\text{in}}/RC$, (2) Compute the frequency response $H(f) = 1/(1+j2\pi fRC)$ using the Fourier Transform, (3) Verify: $ H(f=1/(2\pi RC)) = 1/\sqrt{2}$ — the -3 dB point, (4) Compute the output $y(t) = x(t)*h(t)$ for input $x(t) = \text{rect}(t/T)$ (rectangular pulse) by hand convolution — show the ramp-up and exponential decay, (5) Python: <code>scipy.signal.lfilter</code> with $b=[1/RC]$, $a=[1, 1/RC]$ (digitalised IIR) — verify the impulse response matches $h(t)$. Domain: analog circuit design, separate from CommLink.
Module 4	CO4	Pen + Python	FIR Filter Design by Z-Transform	Students design a 5-tap FIR low-pass filter using the window method (completely different application from CommLink matched filter): (1) Desired frequency response: $H(e^{j\omega}) = 1$ for $ \omega < \pi/4$, 0 for $\pi/4 < \omega < \pi$ — derive the ideal FIR coefficients as the inverse DTFT: $h[n] = (1/2\pi) \int_{-\pi/4}^{\pi/4} H(e^{j\omega})e^{j\omega n} d\omega$ — result: $h[n] = (1/2\pi) \int_{-\pi/4}^{\pi/4} e^{j\omega n} d\omega = \sin(n\pi/4)/(n\pi)$ for $n \neq 0$, $1/4$ for $n=0$, (2) Truncate to 5 taps using rectangular window: $h = [h[-2], h[-1], h[0], h[1], h[2]]$, (3) Compute Z-transform $H(z) = \sum h[n]z^{-n}$ — find zeros of $H(z)$, (4) Python: <code>scipy.signal.freqz</code> to plot the frequency response — verify the passband (0 to $\pi/4$) and stopband. Domain: digital signal processing design, separate from CommLink.
Module 5	CO5	Pen	BER Derivation Practice — QPSK	Students derive the BER for QPSK (Quadrature Phase Shift Keying) by analogy with the BPSK derivation (different modulation from CommLink): QPSK transmits 2 bits per symbol using 4 constellation points at angles $0^\circ, 90^\circ, 180^\circ, 270^\circ$. (1) Define the 4 QPSK symbols as $(\pm A/\sqrt{2})\cos(2\pi f_c t) \pm (A/\sqrt{2})\sin(2\pi f_c t)$, (2) The I and Q components are independent — the probability of bit error in each dimension is $Q(\sqrt{E_b/N_0})$ — not $Q(\sqrt{2E_b/N_0})$ as in BPSK (why?), (3) Derive $P_{e,QPSK} \approx Q(\sqrt{E_b/N_0}) \times 2$ (union bound) — approximately equal to BPSK BER, (4) Compare on the BER curve: QPSK transmits twice the data rate in the same bandwidth as BPSK — at the same BER — making it more spectrally efficient. Derive the spectral efficiency: QPSK = 2 bits/s/Hz vs. BPSK = 1 bit/s/Hz.
Module 6	CO6	Pen + Python	Shannon Capacity Exploration	Students explore Shannon capacity in a different domain (a Wi-Fi channel, not CommLink): A Wi-Fi channel has $B=20$ MHz bandwidth. (1) Plot $C = B \log_2(1+SNR)$ vs. SNR in dB for SNR = 0 to 30 dB — compute C at SNR = 0, 10, 20, 30 dB, (2) 802.11n Wi-Fi uses 64-QAM (6 bits/symbol) at SNR=25 dB — compute the spectral efficiency: 6 bits/symbol $\times$ 1 symbol/s/Hz = 6 bits/s/Hz — compare to Shannon capacity at SNR=25 dB ($C/B \approx 8.6$ bits/s/Hz) — what fraction of capacity is achieved?, (3) Compute the gap between 64-QAM and the Shannon limit — what would

Module	CO	Mode	Micro Project	Description
				LDPC coding (within 0.5 dB of Shannon) provide?, (4) Python: plot the Shannon capacity curve + mark 64-QAM operating point + LDPC-coded operating point. Domain: Wi-Fi communications, separate from CommLink.

TABLE 5 — SOCIAL PROFILE BUILDING

Platform	What to Build	Why It Matters	Expected Impact
GitHub	Repository 'CommLink-BPSK': Jupyter Notebook with all 6 layers + scanned derivation portfolio (photographed hand-written work) + README with the layer-by-layer architecture (each section titled by the barrier it solves). The BER curve plot and Shannon capacity gap plot are the showcase visuals.	A GitHub repository containing a complete BPSK implementation with hand-derived BER formula and Shannon capacity analysis — at Trimester 2 — is exceptional for ECE portfolio. Directly showcases the mathematical-engineering depth required for signal processing, RF, and semiconductor roles.	₹10–22 LPA signal processing, RF engineering, and semiconductor roles; direct differentiator for Qualcomm, MediaTek, Samsung Semiconductor, and ISRO
LinkedIn	Post a LinkedIn article: 'I built a BPSK digital communication system from first principles — here is how each mathematical barrier led me from signal representation to Shannon's theorem'. Include the BER curve (simulated vs. theoretical) as the central visual.	The BER curve — showing simulated bit error rates from Monte Carlo and theoretical $Q(\sqrt{2E_b/N_0})$ overlapping at 6 SNR values — is one of the most compelling technical visuals an ECE student can produce. It demonstrates that the mathematical derivation and the physical simulation agree exactly.	Recruiter visibility at communications companies; ₹10–20 LPA DSP and communications engineering roles
NPTEL	Enrol in NPTEL Communication Systems (IIT Madras/Bombay) simultaneously with MFCS — the course directly reinforces all 6 MFCS modules with IIT depth and provides an exam-based IIT certificate.	The NPTEL certificate + CommLink GitHub portfolio is the strongest communications engineering credential combination available to ECE students in India at Year 1.	IIT-branded communications certificate; ₹10–20 LPA at Jio, Airtel, Nokia, Ericsson, and ISRO

TABLE 6 — CERTIFICATION PATHWAYS

Certification	Issuing Body	Level	Relevance	Notes
NPTEL — Communication Systems (IIT Madras)	IIT Madras (NPTEL)	Foundation — IIT Certificate	Most directly relevant NPTEL course — covers signal representation, Fourier analysis, modulation, noise, and BER — directly mapping to MFCS Modules 1–5. Exam-based IIT certificate.	Free course, paid proctored exam. Enrolling from Week 1 directly reinforces all MFCS modules.
NPTEL — Digital Signal Processing (IIT Kharagpur)	IIT Kharagpur (NPTEL)	Foundation — IIT Certificate	Covers discrete-time signals, Z-transform, FIR/IIR filter design — directly reinforcing Module 4 content. The most comprehensive DSP NPTEL course available.	Strong complement to the CommLink FIR matched filter implementation.
Coursera — Digital Signal Processing (EPFL)	EPFL (Coursera)	Intermediate	EPFL's DSP course covers the Z-transform, filter design, and digital signal processing applications. Directly reinforces MFCS Module 4.	4 weeks directly reinforcing Module 4 content.

TABLE 7 — WEEK-BY-WEEK LESSON PLAN

Week 1 opens with the complete CommLink-BPSK system running — students see the end result before any mathematics is taught. Every subsequent session opens with the CommLink barrier that motivates that session's content.

Week	Module	CO	Session — CommLink barrier exposed, math introduced	CommLink Activity	Micro Project	Tools
Week 1	Module 1	CO1	Day 1: show complete CommLink-BPSK running — BER curve, spectrum, waveform. Remove Layer 1. Ask: 'how do you define the signal mathematically?' Sinusoids introduced. Amplitude, frequency, phase. BPSK symbol definition.	Layer 1: write $s_1(t)$ and $s_0(t)$ fully specified. Generate in Python (NumPy). Plot both waveforms.	Audio Signal Representation micro project begins	Pen + NumPy + Matplotlib
Week 2	Module 1	CO1	Signal energy derivation: $E = \int x^2 dt$. Power. Phasor notation. Orthogonality of sinusoids. Superposition. Sampling theorem preview.	Layer 1 complete: compute $E_b = A^2 T_b / 2$ by hand and verify with NumPy. Plot phasor diagram. Sampling rate specification.	Audio Signal Representation submission	Pen + NumPy
Week 3	Module 2	CO2	Barrier 2: channel bandwidth — what frequencies does BPSK occupy? Fourier Series derivation. Coefficients. Square wave as example. Gibbs phenomenon.	Layer 2 start: Fourier Series of the 1-bit rectangular pulse. Show first 5 harmonics. Python FFT preview.	Square Wave Spectrum Analysis begins	Pen + NumPy FFT
Week 4	Module 2	CO2	Fourier Transform definition. Key pairs (rect $\leftrightarrow$ sinc). Properties: time shift, frequency shift. Parseval's theorem derivation. PSD.	Layer 2 complete: BPSK PSD via FFT. Verify null-to-null bandwidth = $2R$. Parseval check.	Square Wave Spectrum Analysis submission	NumPy FFT + Matplotlib
Week 5	Module 3	CO3	Barrier 3: channel distorts signal. LTI definition: linearity + time-invariance. Impulse response. Convolution integral derivation. Convolution theorem.	Layer 3 start: AWGN channel model. Correlator receiver implementation. Test at $E_b/N_0 = 6$ dB.	RC Low-Pass Filter Analysis begins	Pen + NumPy.correlate
Week 6	Module 3	CO3	Matched filter derivation: maximise SNR $\rightarrow h(t) = s(T_b - t)$. Cauchy-Schwarz proof sketch. Correlator equivalence. $SNR = 2E_b/N_0$.	Layer 3 complete: matched filter correlator for 1,000 bits. Count errors. Verify against Module 5 theory (preview).	RC Low-Pass Filter Analysis submission	NumPy + SciPy
Week 7	Module 4	CO4	Barrier 4: need discrete-time filter. Z-transform definition. Key pairs. Time delay property. Convolution theorem. ROC.	Layer 4 start: FIR matched filter $h[n] = s_1[N-1-n]$. Implement via <code>numpy.convolve</code> . Compare to correlator.	FIR Filter Design begins	Pen + NumPy + SciPy.signal
Week 8	Module 4	CO4	Poles, zeros, unit circle. FIR vs. IIR. Frequency response $ H(e^{j\omega}) $. <code>scipy.signal.freqz</code> . Linear phase FIR condition.	Layer 4 complete: verify FIR matched filter gives identical decisions to correlator for all 1,000 test bits. Plot filter frequency response.	FIR Filter Design submission	SciPy.signal

Week	Module	CO	Session — CommLink barrier exposed, math introduced	CommLink Activity	Micro Project	Tools
Week 9	Module 5	CO5	Barrier 5: how many errors should we expect? AWGN noise model. Gaussian random variable. Q-function. Conditional probability of error.	Layer 5 start: Monte Carlo BER at $E_b/N_0 = 0, 2, 4, 6, 8, 10$ dB. $N=10,000$ bits each.	BER Derivation Practice (QPSK) begins	Pen + NumPy
Week 10	Module 5	CO5	Full BPSK BER derivation: $P(y<0 '1') = Q(\sqrt{2E_b/N_0})$. BER waterfall shape. Theoretical vs. simulated comparison methodology.	Layer 5 complete: Monte Carlo $N=100,000$ bits. Plot simulated BER + theoretical $Q(\sqrt{2E_b/N_0})$ on same log-scale axes.	BER Derivation Practice submission	NumPy + SciPy.special.erfc + Matplotlib
Week 11	Module 6	CO6	Barrier 6: is BPSK near-optimal? Information content. Entropy. Mutual information. Shannon capacity theorem. AWGN capacity: $C=B \log_2(1+SNR)$.	Layer 6: compute Shannon capacity at each CommLink SNR. Plot BPSK spectral efficiency vs. Shannon capacity curve.	Shannon Capacity Exploration begins	Pen + Python
Week 12	Module 6	CO6	Shannon limit computation. Gap analysis. Turbo/LDPC codes. Course synthesis: all 6 layers connected. Viva preparation.	CommLink complete: full Jupyter Notebook with all 6 layers. Shannon gap analysis. Technical summary. Notebook submitted. Viva.	Shannon Capacity Exploration submission + viva	Jupyter Notebook + GitHub

TABLE 8 — LAB EXPERIMENTS (12 LABS)

Labs 1–4 (signal representation + Fourier), Labs 5–6 (convolution + AWGN), Labs 7–8 (Z-transform + FIR), Labs 9–10 (BER derivation + Monte Carlo), Labs 11–12 (Shannon capacity + full integration + viva).

Lab #	Week	CO	Title	Tools	Description & Expected Output
Lab 1	Wk 1	CO1	BPSK Waveform Generation Lab	NumPy + Matplotlib	Students generate both BPSK symbols as NumPy arrays: $f_c=10\text{kHz}$, $R=1\text{kbps}$, $f_s=100\text{kHz}$ (oversampled 100 $\times$). (1) Generate $s_1[n]=\text{numpy.cos}(2\pi \times 10000 \times n / 100000)$ for $n=0..99$ (one bit period), (2) Generate $s_0[n]=-s_1[n]$, (3) Compute energy numerically: $E_b\text{_numerical} = \text{numpy.sum}(s_1^2)/f_s$ — verify against formula $E_b=A^2T_b/2$, (4) Generate a 10-bit random sequence — create the full BPSK waveform by concatenating symbols, (5) Plot: 10-bit waveform showing clean phase reversals at bit transitions. Expected output: 10-bit BPSK waveform plot + energy verification.
Lab 2	Wk 2	CO1	Orthogonality Verification Lab	Pen + NumPy	Students verify sinusoidal orthogonality analytically and numerically: (1) Prove by hand: $\int_0^T \cos(2\pi f_1 t) \cos(2\pi f_2 t) dt = 0$ for $f_1 \neq f_2$ and both integer multiples of $1/T$ — show the product-to-sum identity and integrate each term, (2) Verify numerically: $\text{numpy.sum}(\cos_1 \times \cos_2)/f_s \approx 0$ for $f_1=10\text{kHz}$ and $f_2=20\text{kHz}$ — result should be $< 10^{-10}$ (machine precision), (3) Verify self-orthogonality fails: $\int_0^T \cos^2(2\pi f_1 t) dt = T/2$ — not zero — this is the energy, (4) BPSK implication: the two symbols $s_1=+\cos$ and $s_0=-\cos$ are antipodal (not orthogonal) — compute $\int s_1 s_0 dt = -E_b$ — this minus sign determines the BPSK BER formula. Expected output: pen proof + numerical verification + sign computation.
Lab 3	Wk 3	CO2	FFT Spectrum Lab	NumPy + Matplotlib	Students compute and analyse the BPSK spectrum: (1) Generate 100 random BPSK bits — create the full waveform, (2) numpy.fft.fft on the waveform — numpy.fft.fftfreq for frequency axis, (3) One-sided PSD: $\text{PSD} = \text{FFT} ^2 / (N \times f_s)$ — plot PSD vs. frequency in dB, (4) Mark on the plot: carrier frequency f_c , first nulls at $f_c \pm R$, (5) Parseval verification: compare $\text{numpy.sum}(\text{waveform}^2)/f_s$ (time domain energy) vs. $\text{numpy.sum}(\text{PSD}) \times (f_s/N)$ (frequency domain energy) — must agree within 0.1%, (6) Bandwidth measurement: find the frequencies where PSD drops to 10% of peak — this is the 90% power bandwidth. Expected output: PSD plot with annotations + Parseval numerical verification.
Lab 4	Wk 4	CO2	Fourier Transform Derivation Lab	Pen	Students derive 4 Fourier Transform pairs from scratch: (1) $\text{rect}(t/T)$ — compute $\int_{-T/2}^{T/2} e^{-j2\pi f t} dt$ — result: $T \text{sinc}(fT)$ — show full integral evaluation, (2) $\delta(t)$ — use the sifting property — result: 1 (flat spectrum), (3) $e^{j2\pi f_c t}$ — use the definition — result: $\delta(f-f_c)$, (4) $\cos(2\pi f_c t) = (e^{j2\pi f_c t} + e^{-j2\pi f_c t})/2$ — use linearity and result (3) — result: $(\delta(f-f_c) + \delta(f+f_c))/2$. Apply: the BPSK symbol is $s(t) = A \cos(2\pi f_c t) \times \text{rect}(t/T_b)$ — by the convolution theorem, $S(f) = A/2 \times T_b \text{sinc}((f-f_c)T_b) + A/2 \times T_b \text{sinc}((f+f_c)T_b)$. Draw the spectrum. Expected output: 4 FT derivations + BPSK spectrum sketch.
Lab 5	Wk 5	CO3	Convolution Lab	Pen + NumPy	Students compute convolution analytically and numerically: (1) Hand: compute $(\text{rect}_{\{T\}} * \text{rect}_{\{T\}})(t)$ — the convolution of a rectangle with itself is a triangle — show the 3 cases: $t < 0$, $0 \leq t \leq T$, $T \leq t \leq 2T$, $t > 2T$ — draw the result, (2) NumPy: $\text{numpy.convolve}([1,1,1,1,1], [1,1,1,1,1])$ — verify it produces a triangular shape, (3) BPSK matched filter: the matched filter to $s(t)=A \cos(2\pi f_c t) \times \text{rect}(t/T_b)$ has $h(t) = s(T_b - t)$ — compute $h(t)$ explicitly — it is the time-reversed pulse, (4) Output of matched filter at $t=T_b$: $(s * h)(T_b) = \int_0^{T_b} s(\tau) s(T_b - \tau + T_b) d\tau = \int_0^{T_b} s^2(\tau) d\tau = E_b$ — this is the maximum SNR point. Expected output: triangle convolution derivation + matched filter output at optimal sample time.
Lab 6	Wk 6	CO3	AWGN Channel Simulation Lab	NumPy + Matplotlib	Students implement the complete AWGN channel and correlator: (1) Generate $N=1000$ random BPSK bits, (2) Add AWGN: $\text{noise} = \text{numpy.random.normal}(0, \text{sigma}, \text{len}(\text{signal}))$ where $\text{sigma}=\sqrt{N_0/2}$, (3) Correlator: for each bit period, compute $\text{numpy.sum}(\text{received}[\text{bit_start}:\text{bit_end}] \times \text{reference})$ — sign of

Lab #	Week	CO	Title	Tools	Description & Expected Output
					result = decision, (4) Count errors — compute BER, (5) Repeat for $E_b/N_0 = 2, 4, 6, 8$ dB — collect BER at each SNR, (6) Plot: simulated BER ($N=1000$ bits) — observe high variance (small N), (7) Preview: if N were 100,000 instead of 1,000, the simulated BER would converge to the theoretical curve. Expected output: BER vs. SNR plot (4 points) + error count table.
Lab 7	Wk 7	CO4	Z-Transform Derivation Lab	Pen	Students derive 4 Z-transform pairs from definition: (1) $\delta[n]: \sum_n \delta[n]z^{-n} = z^0 = 1$, (2) $u[n]: \sum_{n=0}^{\infty} z^{-n} = 1/(1-z^{-1})$ for $ z >1$ — geometric series — state ROC, (3) $a^n u[n]: \sum_{n=0}^{\infty} a^n z^{-n} = 1/(1-az^{-1})$ for $ z > a $, (4) The BPSK FIR matched filter $h[n]=\{h_0, h_1, \dots, h_{N-1}\}$ — its Z-transform $H(z) = \sum_n h_n z^{-n}$ — the zeros of $H(z)$ are at z values satisfying $H(z)=0$ — compute numerically for a 5-tap filter. For each: verify the region of convergence and state whether the sequence is causal. Expected output: 4 Z-transform derivations with ROC statements + $H(z)$ for 5-tap filter.
Lab 8	Wk 8	CO4	FIR Filter Frequency Response Lab	SciPy + NumPy + Matplotlib	Students implement and characterise the CommLink FIR matched filter: (1) Generate the matched filter coefficients: $h = \text{numpy.flipud}(s_t[0:\text{samples_per_bit}])$ (time-reversed symbol), (2) Plot: filter coefficients $h[n]$ vs. n , (3) Frequency response: $\text{scipy.signal.freqz}(h, \text{worN}=1024)$ — plot $ H(e^{j\omega}) $ vs. frequency — mark the passband centred at $f_c/f_s \times 2\pi$, (4) Apply FIR filter: $\text{output} = \text{numpy.convolve}(\text{received_signal}, h)$ — sample at $n=\text{samples_per_bit}, 2 \times \text{samples_per_bit}$, etc. for each bit decision, (5) Verify: FIR filter decisions must be identical to correlator decisions for 100 test bits (they are mathematically equivalent), (6) Compare 3 filter lengths ($N=10, 50, 100$ samples per bit) — which gives the cleanest BER? Expected output: frequency response plot + equivalence verification + BER vs. filter length table.
Lab 9	Wk 9	CO5	Q-Function and BER Lab	Pen + SciPy	Students derive and evaluate the BPSK BER: (1) Pen: full derivation of $P_e = Q(\sqrt{2E_b/N_0})$ — starting from received sample $y = +\sqrt{E_b} + n$, $n \sim N(0, N_0/2)$ — every step shown: $P(\text{error} '1') = P(y < 0) = P(n < -\sqrt{E_b}) = P(n/\sqrt{N_0/2} < -\sqrt{2E_b/N_0}) = Q(\sqrt{2E_b/N_0})$, (2) Evaluate $Q(x) = \text{scipy.special.erfc}(x/\sqrt{2})/2$ at $x=\sqrt{2E_b/N_0}$ for $E_b/N_0 = 0.2, 4, 6, 8, 10$ dB — tabulate, (3) Plot the BER vs. E_b/N_0 on a log-scale (theoretical only at this stage), (4) At $E_b/N_0=6$ dB: $P_e \approx 0.0079$ — how many errors in 10,000 bits? (≈ 79) — verify this matches intuition from Lab 6. Expected output: full BER derivation on paper + theoretical BER table + BER curve plot.
Lab 10	Wk 10	CO5	Monte Carlo BER Validation Lab	NumPy + SciPy + Matplotlib	Students produce the definitive CommLink BER curve: for $E_b/N_0 = \{0.2, 4, 6, 8, 10\}$ dB, with $N=100,000$ bits per SNR point: (1) Compute $\sigma^2 = E_b / (2 \times 10^{\{\text{SNR_dB}/10\}})$, (2) Generate 100,000 BPSK bits, transmit through AWGN with this σ , apply FIR matched filter, count errors, (3) Theoretical BER: $\text{scipy.special.erfc}(\text{numpy.sqrt}(10^{\{\text{SNR_dB}/10\}})) / 2$, (4) Plot: simulated BER (circles) and theoretical BER (solid line) on same log-scale axes — both curves must overlap within $\pm 20\%$ at all SNR points, (5) Annotate the plot: mark $E_b/N_0=9.6$ dB (BER= 10^{-5} target) on the theoretical curve. Expected output: BER curve plot with simulated + theoretical + SNR target annotated.
Lab 11	Wk 11	CO6	Shannon Capacity Computation Lab	Pen + Python	Students compute the CommLink Shannon capacity analysis: (1) Pen: derive $C = B \log_2(1+\text{SNR})$ for AWGN channel — sketch the capacity proof: Gaussian inputs maximise entropy for fixed power — entropy argument, (2) CommLink parameters: $B=2R=2000$ Hz (null-to-null), $\text{SNR} = E_b/N_0 \times (R/B) = E_b/N_0 / 2$, (3) Python: plot $C(\text{SNR_dB})/B$ in bits/s/Hz from $\text{SNR}=-10$ to 30 dB — this is the maximum achievable spectral efficiency, (4) Mark: BPSK operating point (1 bit/s/Hz horizontal line) and the SNR where it crosses the capacity curve (-1.59 dB — the Shannon limit), (5) Mark: BPSK actual operating point at BER= 10^{-5} ($E_b/N_0=9.6$ dB) — show the 11.2 dB gap to the Shannon limit. Expected output: capacity vs. spectral efficiency plot + gap annotated.

Lab #	Week	CO	Title	Tools	Description & Expected Output
Lab 12	Wk 12	CO1–CO6	CommLink Full Integration + Viva Lab	Jupyter Notebook	Students submit the complete CommLink-BPSK Jupyter Notebook: Section 1 (Layer 1 — waveform), Section 2 (Layer 2 — spectrum), Section 3 (Layer 3 — channel + correlator), Section 4 (Layer 4 — FIR matched filter), Section 5 (Layer 5 — BER curve), Section 6 (Layer 6 — Shannon capacity). Each section starts with the CommLink barrier it solves. Faculty viva (10 minutes): selects one CommLink layer at random — student derives the key mathematical result on blank paper, explains the barrier, demonstrates the Python cell. Expected output: complete Jupyter Notebook submitted + GitHub repository published + viva completed.

TABLE 9 — REFERENCE BOOKS

Book Title	Author(s)	Publisher	Year	Relevance
Communication Systems	Simon Haykin & Michael Moher	Wiley	2009 (4th Ed)	The global standard communications textbook. Modules 1–5 map directly to Haykin's chapters on signal representation, Fourier analysis, noise, modulation, and BER. The BPSK BER derivation in Chapter 8 is the authoritative reference.
Signals and Systems	Alan V. Oppenheim & Alan S. Willsky	Pearson	1997 (2nd Ed)	The global standard signals and systems textbook. Modules 1–4 draw from this book: sinusoidal signals, Fourier Series, Fourier Transform, and LTI systems. The derivations match MFCS's treatment exactly.
Discrete-Time Signal Processing	Alan V. Oppenheim & Ronald W. Schafer	Pearson	2009 (3rd Ed)	The definitive discrete-time signal processing reference. Module 4 (Z-transform, FIR/IIR design) draws directly from this book.
Elements of Information Theory	Thomas M. Cover & Joy A. Thomas	Wiley	2006 (2nd Ed)	The definitive information theory textbook. Shannon's channel capacity theorem, entropy, and mutual information (Module 6) are derived here with full mathematical rigour.
Think DSP: Digital Signal Processing in Python	Allen Downey	O'Reilly Media	2016 (Free at greenteapress.com)	Python-first DSP textbook — FFT, filtering, and signal analysis with NumPy. Directly supports MFCS Modules 2–4 implementation. Free PDF.

TABLE 10 — MOOC RECOMMENDATIONS

Course Title	Type	Provider	Relevance	Link
Communication Systems (IIT Madras — NPTEL)	Free + IIT Cert	IIT Madras (NPTEL)	Most directly aligned MOOC — covers BPSK, AWGN, BER, Fourier analysis — maps to all 6 MFCS modules.	https://nptel.ac.in/courses/117106086
Digital Signal Processing (EPFL — Coursera)	Course	EPFL (Coursera)	Z-transform, filter design, and digital signal processing — directly reinforces Module 4.	https://www.coursera.org/learn/dsp
Information Theory (MIT OCW 6.441)	Free Course	MIT OpenCourseWare	Entropy, mutual information, channel capacity — directly extends Module 6.	https://ocw.mit.edu/courses/6-441-information-theory-spring-2016/
Think DSP (Interactive)	Free Book	Allen Downey	Python-first DSP — FFT, filtering, and signal analysis. Primary Python companion for Modules 2–4.	https://greenteapress.com/wp/think-dsp/

— End of Course Handout: Mathematics for Communication Systems (MFCS) — PBL REBUILD —